CBC Parameter Reference

COIN-OR Cbc Development Team

July 2026 — CBC devel

Contents

1 Stopping **2**
-allowableGap . . . 2
-cutoff . . . 2
-ratioGap . . . 2
-maxNodes . . . 2
-maxNNIFS . . . 2
-secnifs . . . 3
-maxSolutions . . . 3
-seconds . . . 3
-lpseconds . . . 3

2 MIP Preprocessing **3**
-PrepNames . . . 3
-sosOptions . . . 3
-clqstrengthen . . . 4
-preprocess . . . 4
-cppGenerate . . . 4
-extraVariables . . . 4
-tunePreProcess . . . 4
-fixOnDj . . . 4
-tightenFactor . . . 5

3 MIP Preprocessing — Bound Propagation **5**
-doBoundPropagation . . . 5
-singletonBounds . . . 5
-boundPropLevel . . . 5
-nodeBoundProp . . . 5
-boundPropMaxRounds . . . 6
-nodeBoundPropMaxDepth . . . 6
-nodeBoundPropMinDepth . . . 6
-nodeBoundPropDepthInterval . . . 6

4 LP Presolve **6**
-presolve . . . 6
-passPresolve . . . 6
-preTolerance . . . 7

5 Cuts **7**
-cliqueCuts . . . 7

-cutsOnOff . . . 7
-flowCoverCuts . . . 7
-GMICuts . . . 7
-gomoryCuts . . . 8
-knapsackCuts . . . 8
-lagomoryCuts . . . 8
-liftAndProjectCuts . . . 8
-latwomirCuts . . . 9
-mixedIntegerRoundingCuts . . . 9
-oddwheelCuts . . . 9
-probingCuts . . . 9
-reduceAndSplitCuts . . . 9
-reduce2AndSplitCuts . . . 10
-residualCapacityCuts . . . 10
-twoMirCuts . . . 10
-zeroHalfCuts . . . 10
-aggregatelevel . . . 10
-cutDepth . . . 11
-cutLength . . . 11
-passTreeCuts . . . 11
-slowcutpasses . . . 11
-zeroHalfRowMaxFractionalCount . . . 11
-zeroHalfRowMaxPairCount . . . 11
-zeroHalfSparseThreshold . . . 12
-passCuts . . . 12

6 Heuristics **12**
6.1 Constructive Heuristics . . . 12
-DivingCoefficient . . . 12
-DivingFractional . . . 12
-DivingGuided . . . 12
-DivingLineSearch . . . 13
-DivingPseudocost . . . 13
-DivingSome . . . 13
-DivingVectorLength . . . 13
-feasibilityPump . . . 14
-greedyHeuristic . . . 14
-naiveHeuristics . . . 14
-pivotAndFix . . . 14
-randomizedRounding . . . 14
-Rens . . . 15
-roundingHeuristic . . . 15
6.2 Improvement Heuristics . . . 15
-Dins . . . 15
-dwHeuristic . . . 15
-localTreeSearch . . . 15
-proximitySearch . . . 16
-Rins . . . 16
-VndVariableNeighborhoodSearch . . . 16
6.3 Improvement Heuristics (2+ solutions) . . . 16
-combineSolutions . . . 16
-combine2Solutions . . . 17

6.4 General Heuristic Settings . . . 17
-heuristicsOnOff . . . 17
-doHeuristic . . . 17
-forceSolution . . . 17
-depthMiniBab . . . 17
-diveOpt . . . 18
-diveSolves . . . 18
-passFeasibilityPump . . . 18
-pumpTune . . . 18
-hOptions . . . 18
-fpumpPassFreq . . . 18
-artificialCost . . . 19
-rinsCloseMaxDist . . . 19

7 Branching **19**
-branchPriorities . . . 19
-nodeStrategy . . . 19
-OrbitalBranching . . . 19
-sosPrioritize . . . 20
-trustPseudocosts . . . 20
-strongBranching . . . 20

8 Tolerances **20**
-increment . . . 20
-infeasibilityWeight . . . 20
-integerTolerance . . . 20
-dualTolerance . . . 21
-primalTolerance . . . 21
-zeroTolerance . . . 21

9 Conflict Graph **21**
-cgraph . . . 21
-bkpivoting . . . 21
-bkmaxcalls . . . 21
-bkclqextmethod . . . 22
-oddwextmethod . . . 22

10 Strategy **22**
-strategy . . . 22
-experiment . . . 22
-hotStartMaxIts . . . 22
-multipleRootPasses . . . 22
-options . . . 23
-pumpCutoff . . . 23
-pumpIncrement . . . 23
-fractionforBAB . . . 23
-fakeBound . . . 23

11 Solving **23**
-solve . . . 23
-initialSolve . . . 24
-strengthen . . . 24

-constraintfromCutoff . . . 24
-lpMethod . . . 24
-maxSavedSolutions . . . 24
-randomCbcSeed . . . 25
-direction . . . 25
-maximize . . . 25
-minimize . . . 25
-allSlack . . . 25
-barrier . . . 25
-dualSimplex . . . 25
-eitherSimplex . . . 26
-guess . . . 26
-network . . . 26
-parametrics . . . 26
-plusMinus . . . 26
-primalSimplex . . . 26
-reallyScale . . . 27
-reverse . . . 27
-direction . . . 27
-vector . . . 27
-decompose . . . 27
-dualize . . . 27
-randomSeed . . . 27

12 Simplex **28**
-KKT . . . 28
-perturbation . . . 28
-crash . . . 28
-dualPivot . . . 28
-factorization . . . 28
-primalPivot . . . 28
-denseThreshold . . . 29
-idiotCrash . . . 29
-maxFactor . . . 29
-maxIterations . . . 29
-moreSpecialOptions . . . 29
-pertValue . . . 29
-sprintCrash . . . 30
-slpValue . . . 30
-smallFactorization . . . 30
-specialOptions . . . 30
-dualBound . . . 30
-primalWeight . . . 30
-psi . . . 31

13 Barrier **31**
-cholesky . . . 31
-crossover . . . 31
-gamma(Delta) . . . 31

14 Scaling **31**
-scaling . . . 31

-objectiveScale . . . 32
-reallyObjectiveScale . . . 32
-rhsScale . . . 32

15 Output **32**
-statistics . . . 32
-printMask . . . 32
-precisionOutput . . . 32
-messages . . . 33
-checktimeFrequency . . . 33
-printingOptions . . . 33
-timeMode . . . 33
-logLevel . . . 33
-lplogLevel . . . 33
-flushPerNewLine . . . 34
-useUTF8 . . . 34
-compactTables . . . 34
-lpIterFreq . . . 34
-outputFormat . . . 34
-pOptions . . . 35
-lpTimeFreq . . . 35
-fpumpTimeFreq . . . 35
-bufferedMode . . . 35
-messages . . . 35
-allCommands . . . 35
-printingOptions . . . 35
-cppGenerate . . . 36
-progressInterval . . . 36

16 I/O **36**
-export . . . 36
-import . . . 36
-printSolution . . . 36
-mipStart . . . 36
-readPriorities . . . 37
-readModel . . . 37
-writeGSolution . . . 37
-writeModel . . . 37
-nextBestSolution . . . 37
-writeSolution . . . 38
-solution . . . 38
-writeSolBinary . . . 38
-writeStatistics . . . 38
-writeFeatures . . . 38
-checkSolution . . . 38
-csvFeatures . . . 39
-csvStatistics . . . 39
-exportFile . . . 39
-importFile . . . 39
-gmplSolutionFile . . . 39
-mipReadFile . . . 39
-modelFile . . . 39

-nextSolutionFile . . . 39
-priorityFile . . . 40
-solFile . . . 40
-solBinaryFile . . . 40
-directory . . . 40
-dirNetlib . . . 40
-errorsAllowed . . . 40
-basisIn . . . 40
-basisOut . . . 41
-basisFile . . . 41
-paramFile . . . 41
-errorsAllowed . . . 41
-keepNames . . . 41

17 Parallelism **42**
-racingLP . . . 42
-threads . . . 42

18 General **42**
-help . . . 42
-end . . . 42
-exit . . . 42
-quit . . . 42
-stop . . . 42
-version . . . 43
-cplexUse . . . 43
-rankConflictType . . . 43
-extra1 . . . 43
-extra2 . . . 43
-extra3 . . . 43
-extra4 . . . 43
-rootHeurSchedule . . . 43
-rankConflict . . . 44
-rankConflictPowerTrusted . . . 44
-rankConflictPowerUntrusted . . . 44
-rankRange . . . 44
-rankRangePowerTrusted . . . 44
-rankRangePowerUntrusted . . . 45
-rankRangeMax . . . 45
-rankObjCoeff . . . 45
-rankObjCoeffPowerTrusted . . . 45
-rankObjCoeffPowerUntrusted . . . 45
-rankNonzeros . . . 45
-rankNonzerosPowerTrusted . . . 46
-rankNonzerosPowerUntrusted . . . 46
-rankConflictMaxPercBin . . . 46
-netlibBarrier . . . 46
-netlibDual . . . 46
-netlib . . . 46
-netlibPrimal . . . 47
-netlibTune . . . 47

Introduction

CBC (COIN-OR Branch and Cut) is an open-source mixed-integer programming solver. This document provides a complete reference for all command-line parameters, organized by functional topic.

Parameters are specified on the command line *before* `-solve`:

```
cbc model.mps -sec 300 -cuts ifmove -solve
```

Both single-dash (`-sec`) and double-dash (`-sec`) styles are accepted. In interactive mode, omit the leading dash.

1 Stopping

`-allowableGap`

Stop when gap between best possible and incumbent is less than this

If the gap between best solution and best possible solution is less than this then the search will be terminated. Also see ratioGap.

Range: 0 to ∞ (default: 1e-12)

`-cutoff`

All solutions must be better than this

All solutions must be better than this value (in a minimization sense). This is also set by cbc whenever it obtains a solution and is set to the value of the objective for the solution minus the cutoff increment.

Range: $-\infty$ to ∞ (default: 1e+50)

`-ratioGap`

Stop when the gap between the best possible solution and the incumbent is less than this fraction of the larger of the two

If the gap between the best solution and the best possible solution is less than this fraction of the objective value at the root node then the search will terminate. See 'allowableGap' for a way of using absolute value rather than fraction.

Range: 0 to ∞ (default: 0)

`-maxNodes`

Maximum number of nodes to evaluate

This is a repeatable way to limit search. Normally using time is easier but then the results may not be repeatable.

Range: 0 to INT_MAX (default: 2147483647)

`-maxNNIFS`

Maximum number of nodes to be processed without improving the incumbent solution.

This criterion specifies that when a feasible solution is available, the search should continue only if better feasible solutions were produced in the last nodes.

Range: -1 to INT_MAX (default: 2147483647)

`-secnifs`

maximum seconds without improving the incumbent solution

With this stopping criterion, after a feasible solution is found, the search should continue only if the incumbent solution was updated recently, the tolerance is specified here.

Range: -1 to inf (default: inf)

`-maxSolutions`

Maximum number of feasible solutions to get

You may want to stop after (say) two solutions or an hour. This is checked every node in tree, so it is possible to get more solutions from heuristics.

Range: 1 to 1073741823 (default: 1073741823)

`-seconds`

Maximum seconds for branch and cut

After this many seconds the program will act as if maximum nodes had been reached. You may wish to also set '-check less' which stops cbc checking time quite as often which reduces system time.

Range: -1 to 1000000000000 (default: 100000000)

`-lpseconds`

Maximum seconds

After this many seconds clp will act as if maximum iterations had been reached (if value >=0).

Range: -1 to inf (default: -1)

2 MIP Preprocessing

`-PrepNames`

If column names will be kept in pre-processed model

Normally the preprocessed model has column names replaced by new names C0000... Setting this option to on keeps original names in variables which still exist in the preprocessed problem

Values: `off`, `on` (default: `on`)

`-sosOptions`

Whether to use SOS from AMPL

Normally if AMPL says there are SOS variables they should be used, but sometimes they should be turned off - this does so.

Values: `off`, `on` (default: `off`)

-clqstrengthen

Whether and when to perform Clique Strengthening preprocessing routine

Values: `off`, `before`, `after` (default: `after`)

-preprocess

Whether to use integer preprocessing

This tries to reduce size of the model in a similar way to presolve and it also tries to strengthen the model. This can be very useful and is worth trying. save option saves on file presolved.mps. equal will turn <= cliques into ==. sos will create sos sets if all 0-1 in sets (well one extra is allowed) and no overlaps. trysos is same but allows any number extra. equalall will turn all valid inequalities into equalities with integer slacks. strategy is as on but uses CbcStrategy.

Values: `off`, `on`, `save`, `equal`, `sos`, `trysos`, `equalall`, `strategy`, `aggregate`, `forcesos`, `stop!aftersaving`, `equalallstop` (default: `sos`)

-cppGenerate

Generates C++ code

Once you like what the stand-alone solver does then this allows you to generate user_driver.cpp which approximates the code. 0 gives simplest driver, 1 generates saves and restores, 2 generates saves and restores even for variables at default value. 4 bit in cbc generates size dependent code rather than computed values.

Range: 0 to 4 (default: 0)

-extraVariables

Allow creation of extra integer variables

Switches on a trivial re-formulation that introduces extra integer variables to group together variables with same cost.

Range: −INT_MAX to INT_MAX (default: 0)

-tunePreProcess

Dubious tuning parameters for preprocessing

Format aabbcccc - If aa then this is number of major passes (i.e. with presolve) If bb and bb>0 then this is number of minor passes (if unset or 0 then 10) cccc is bit set 0 - 1 Heavy probing 1 - 2 Make variables integer if possible (if obj value) 2 - 4 As above but even if zero objective value 7 - 128 Try and create cliques 8 - 256 If all +1 try hard for dominated rows 9 - 512 Even heavier probing 10 - 1024 Use a larger feasibility tolerance in presolve 11 - 2048 Try probing before creating cliques 12 - 4096 Switch off duplicate column checking for integers 13 - 8192 Allow scaled duplicate column checking

Now aa 99 has special meaning i.e. just one simple presolve.

Range: 0 to INT_MAX (default: 7)

-fixOnDj

Try heuristic that fixes variables based on reduced costs

If set, integer variables with reduced costs greater than the specified value will be fixed before branch and bound - use with extreme caution!

Range: $-\infty$ to ∞ (default: 0)

`-tightenFactor`

Tighten bounds using value times largest activity at continuous solution

This sleazy trick can help on some problems.

Range: 0 to inf (default: 0)

3 MIP Preprocessing — Bound Propagation

`-doBoundPropagation`

Run bound propagation on the loaded model

Immediately runs bound propagation on the currently loaded model, applying bound tightenings to the problem in place. The aggression level is controlled by boundPropLevel. After running, use writeModel to save the tightened problem.

`-singletonBounds`

Whether to tighten variable bounds from singleton rows before solve

When on, singleton rows (rows with a single nonzero) are used to tighten variable bounds before the initial LP solve and conflict graph construction. This is a cheap preprocessing step that can fix variables and reduce the problem size.

Values: `off`, `on` (default: `on`)

`-boundPropLevel`

Aggression level for bound propagation before solve

Controls how aggressively bound propagation tightens variable bounds before the initial LP solve. off: disabled (falls back to singletonBounds setting). singletons: singleton rows only — same as singletonBounds on. milpbt: singletons then knapsack-based bound propagation for up to boundPropMaxRounds rounds (default 100, effectively fixpoint). fixpoint: singletons then bound propagation until no new fixings are found, regardless of boundPropMaxRounds.

Values: `off`, `singletons`, `milpbt`, `fixpoint` (default: `milpbt`)

`-nodeBoundProp`

Run bound propagation at B&B nodes

When enabled, runs knapsack-based bound propagation after branching decisions are applied at each node (subject to depth constraints), before the LP is solved. Can detect infeasibility earlier and fix additional variables. Controlled by nodeBoundPropMaxDepth and nodeBoundPropDepthInterval.

Values: `off`, `on` (default: `off`)

`-boundPropMaxRounds`

Maximum number of bound propagation rounds

Maximum number of CoinBoundPropagation rounds when boundPropLevel is 'milpbt'. Each round re-examines all rows using the bounds fixed in previous rounds; the process stops early if a round produces no new fixings. Has no effect when boundPropLevel is 'fixpoint' (runs until fixpoint regardless) or 'off'/'singletons'.

Range: 1 to INT_MAX (default: 100)

`-nodeBoundPropMaxDepth`

Maximum tree depth at which node bound propagation is applied

Node bound propagation is only applied at depths up to this value. Deeper nodes skip bound propagation to reduce overhead.

Range: 0 to INT_MAX (default: 50)

`-nodeBoundPropMinDepth`

Minimum tree depth at which node bound propagation is applied

Node bound propagation is only applied at depths at or above this value. Shallower nodes skip bound propagation.

Range: 0 to INT_MAX (default: 5)

`-nodeBoundPropDepthInterval`

Depth interval for node bound propagation

Node bound propagation is applied at depths that are multiples of this interval (0, interval, 2*interval, ...). For example, with interval 3 bound propagation runs at depths 0, 3, 6, 9, etc.

Range: 1 to INT_MAX (default: 5)

4 LP Presolve

`-presolve`

Whether to presolve problem

Presolve analyzes the model to find such things as redundant equations, equations which fix some variables, equations which can be transformed into bounds, etc. For the initial solve of any problem this is worth doing unless one knows that it will have no effect. Option 'on' will normally do 5 passes, while using 'more' will do 10. If the problem is very large one can let CLP write the original problem to file by using 'file'.

Values: `on`, `off`, `more`, `file` (default: `on`)

`-passPresolve`

How many passes in presolve

Normally Presolve does 10 passes but you may want to do less to make it more lightweight or do more if improvements are still being made. As Presolve will return if nothing is being taken out, you should not normally need to use this fine tuning.

Range: -200 to 100 (default: 5)

`-preTolerance`

Tolerance to use in presolve

One may want to increase this tolerance if presolve says the problem is infeasible and one has awkward numbers and is sure that the problem is really feasible.

Range: 1e-20 to inf (default: 1e-08)

5 Cuts

`-cliqueCuts`

Whether to use clique cuts

This switches on clique cuts (either at root or in entire tree). An improved version of the Bron-Kerbosch algorithm is used to separate cliques.

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `onglobal` (default: `ifmove`)

`-cutsOnOff`

Switches all cuts on or off

This can be used to switch on or off all cuts (apart from Reduce and Split). Then you can set individual ones off or on. See branchAndCut for information on options.

Values: `off`, `on`, `root`, `ifmove`, `forceon` (default: `on`)

`-flowCoverCuts`

Whether to use Flow Cover cuts

Value 'on' enables the cut generator and CBC will try it in the branch and cut tree (see cutDepth on how to fine tune the behavior). Value 'root' lets CBC run the cut generator generate only at the root node. Value 'ifmove' lets CBC use the cut generator in the tree if it looks as if it is doing some good and moves the objective value. Value 'forceon' turns on the cut generator and forces CBC to use it at every node. Reference: https://github.com/coin-or/Cgl/wiki/CglFlowCover

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `onglobal` (default: `ifmove`)

`-GMICuts`

Whether to use alternative Gomory cuts

Value 'on' enables the cut generator and CBC will try it in the branch and cut tree (see cutDepth on how to fine tune the behavior). Value 'root' lets CBC run the cut generator generate only at the root node. Value 'ifmove' lets CBC use the cut generator in the tree if it looks as if it is doing some good and moves the objective value. Value 'forceon' turns on the cut generator and forces CBC to use it at every node. This version is by Giacomo Nannicini and may be more robust than gomoryCuts.

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `endonly`, `long`, `longroot`, `longifmove`, `forcelongon`, `longendonly` (default: `off`)

-gomoryCuts

Whether to use Gomory cuts

The original cuts - beware of imitations! Having gone out of favor, they are now more fashionable as LP solvers are more robust and they interact well with other cuts. They will almost always give cuts (although in this executable they are limited as to number of variables in cut). However the cuts may be dense so it is worth experimenting (Long allows any length). Value 'on' enables the cut generator and CBC will try it in the branch and cut tree (see cutDepth on how to fine tune the behavior). Value 'root' lets CBC run the cut generator generate only at the root node. Value 'ifmove' lets CBC use the cut generator in the tree if it looks as if it is doing some good and moves the objective value. Value 'forceon' turns on the cut generator and forces CBC to use it at every node. Reference: https://github.com/coin-or/Cgl/wiki/CglGomory

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `forceandglobal`, `forcelongon`, `onglobal`, `longer`, `shorter` (default: `ifmove`)

-knapsackCuts

Whether to use Knapsack cuts

Value 'on' enables the cut generator and CBC will try it in the branch and cut tree (see cutDepth on how to fine tune the behavior). Value 'root' lets CBC run the cut generator generate only at the root node. Value 'ifmove' lets CBC use the cut generator in the tree if it looks as if it is doing some good and moves the objective value. Value 'forceon' turns on the cut generator and forces CBC to use it at every node. Reference: https://github.com/coin-or/Cgl/wiki/CglKnapsackCover

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `forceandglobal`, `onglobal` (default: `ifmove`)

-lagomoryCuts

Whether to use Lagrangean Gomory cuts

This is a gross simplification of 'A Relax-and-Cut Framework for Gomory's Mixed-Integer Cuts' by Matteo Fischetti & Domenico Salvagnin. This simplification just uses original constraints while modifying objective using other cuts. So you don't use messy constraints generated by Gomory etc. A variant is to allow non messy cuts e.g. clique cuts. So 'only' does this while 'clean' also allows integral valued cuts. 'End' is recommended and waits until other cuts have finished before it does a few passes. The length options for gomory cuts are used.

Values: `off`, `root`, `endonly`, `endonlyroot`, `endclean`, `endcleanroot`, `endboth`, `onlyaswell`, `onlyaswellroot`, `cleanaswell`, `cleanaswellroot`, `bothaswell`, `bothaswellroot`, `onlyinstead`, `cleaninstead`, `bothinstead` (default: `off`)

-liftAndProjectCuts

Whether to use lift-and-project cuts

These cuts may be expensive to compute. Value 'on' enables the cut generator and CBC will try it in the branch and cut tree (see cutDepth on how to fine tune the behavior). Value 'root' lets CBC run the cut generator generate only at the root node. Value 'ifmove' lets CBC use the cut generator in the tree if it looks as if it is doing some good and moves the objective value. Value 'forceon' turns on the cut generator and forces CBC to use it at every node. Reference: https://github.com/coin-or/Cgl/wiki/CglLandP

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `iflongon` (default: `off`)

-latwomirCuts

Whether to use Lagrangean Twomir cuts

This is a Lagrangean relaxation for Twomir cuts. See lagomoryCuts for description of options.

Values: `off`, `endonly`, `endonlyroot`, `endclean`, `endcleanroot`, `endboth`, `onlyaswell`, `cleanaswell`, `bothaswell`, `onlyinstead`, `cleaninstead`, `bothinstead` (default: `off`)

-mixedIntegerRoundingCuts

Whether to use Mixed Integer Rounding cuts

Value 'on' enables the cut generator and CBC will try it in the branch and cut tree (see cutDepth on how to fine tune the behavior). Value 'root' lets CBC run the cut generator generate only at the root node. Value 'ifmove' lets CBC use the cut generator in the tree if it looks as if it is doing some good and moves the objective value. Value 'forceon' turns on the cut generator and forces CBC to use it at every node. Reference: https://github.com/coin-or/Cgl/wiki/CglMixedIntegerRounding2

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `onglobal` (default: `ifmove`)

-oddwheelCuts

Whether to use odd wheel cuts

This switches on odd-wheel inequalities (either at root or in entire tree).

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `onglobal` (default: `off`)

-probingCuts

Whether to use Probing cuts

Value 'forceOnBut' turns on probing and forces CBC to do probing at every node, but does only probing, not strengthening etc. Value 'strong' forces CBC to strongly do probing at every node, that is, also when CBC would usually turn it off because it hasn't found something. Value 'forceonbutstrong' is like 'forceonstrong', but does only probing (column fixing) and turns off row strengthening, so the matrix will not change inside the branch and bound.Reference: https://github.com/coin-or/Cgl/wiki/CglProbing

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `forceonbut`, `forceonbutstrong`, `forceonglobal`, `forceonstrong`, `onglobal`, `strongroot` (default: `ifmove`)

-reduceAndSplitCuts

Whether to use Reduce-and-Split cuts

These cuts may be expensive to generate. Value 'on' enables the cut generator and CBC will try it in the branch and cut tree (see cutDepth on how to fine tune the behavior). Value 'root' lets CBC run the cut generator generate only at the root node. Value 'ifmove' lets CBC use the cut generator in the tree if it looks as if it is doing some good and moves the objective value. Value 'forceon' turns on the cut generator and forces CBC to use it at every node. Reference: https://github.com/coin-or/Cgl/wiki/CglRedSplit

Values: `off`, `on`, `root`, `ifmove`, `forceon` (default: `off`)

-reduce2AndSplitCuts

Whether to use Reduce-and-Split cuts - style 2

This switches on reduce and split cuts (either at root or in entire tree). This version is by Giacomo Nannicini based on Francois Margot's version. Standard setting only uses rows in tableau <= 256, long uses all. These cuts may be expensive to generate. See option cuts for more information on the possible values.

Values: `off`, `on`, `root`, `longon`, `longroot` (default: `off`)

-residualCapacityCuts

Whether to use Residual Capacity cuts

Value 'on' enables the cut generator and CBC will try it in the branch and cut tree (see cutDepth on how to fine tune the behavior). Value 'root' lets CBC run the cut generator generate only at the root node. Value 'ifmove' lets CBC use the cut generator in the tree if it looks as if it is doing some good and moves the objective value. Value 'forceon' turns on the cut generator and forces CBC to use it at every node. Reference: https://github.com/coin-or/Cgl/wiki/CglResidualCapacity

Values: `off`, `on`, `root`, `ifmove`, `forceon` (default: `off`)

-twoMirCuts

Whether to use Two phase Mixed Integer Rounding cuts

Value 'on' enables the cut generator and CBC will try it in the branch and cut tree (see cutDepth on how to fine tune the behavior). Value 'root' lets CBC run the cut generator generate only at the root node. Value 'ifmove' lets CBC use the cut generator in the tree if it looks as if it is doing some good and moves the objective value. Value 'forceon' turns on the cut generator and forces CBC to use it at every node. Reference: https://github.com/coin-or/Cgl/wiki/CglTwomir

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `forceandglobal`, `forcelongon`, `onglobal` (default: `ifmove`)

-zeroHalfCuts

Whether to use zero half cuts

Value 'on' enables the cut generator and CBC will try it in the branch and cut tree (see cutDepth on how to fine tune the behavior). Value 'root' lets CBC run the cut generator generate only at the root node. Value 'ifmove' lets CBC use the cut generator in the tree if it looks as if it is doing some good and moves the objective value. Value 'forceon' turns on the cut generator and forces CBC to use it at every node. This implementation was written by Alberto Caprara.

Values: `off`, `on`, `root`, `ifmove`, `forceon`, `onglobal` (default: `ifmove`)

-aggregatelevel

Level of aggregation used in CglMixedRounding

MixedIntegerRounding2 can work on constraints created by aggregating constraints in model. Although the coding for this has been in for some time, it is being modified and the user may wish to play with this. -1 varies the level at various times.

Range: -1 to 5 (default: 1)

-cutDepth

Depth in tree at which to do cuts

Cut generators may be off, on only at the root, on if they look useful, and on at some interval. If they are done every node then that is that, but it may be worth doing them every so often. The original method was every so many nodes but it is more logical to do it whenever depth in tree is a multiple of K. This option does that and defaults to -1 (off).

Range: -1 to INT_MAX (default: -1)

-cutLength

Length of a cut

At present this only applies to Gomory cuts. -1 (default) leaves as is. Any value >0 says that all cuts <= this length can be generated both at root node and in tree. 0 says to use some dynamic lengths. If value >=10,000,000 then the length in tree is value%10000000 - so 10000100 means unlimited length at root and 100 in tree.

Range: -1 to INT_MAX (default: -1)

-passTreeCuts

Number of rounds that cut generators are applied in the tree

The default is to do one pass. A negative value -n means that n passes are also applied if the objective does not drop.

Range: −INT_MAX to INT_MAX (default: 10)

-slowcutpasses

Maximum number of rounds for slower cut generators

Some cut generators are fairly slow - this limits the number of times they are tried. The cut generators identified as 'may be slow' at present are Lift and project cuts and both versions of Reduce and Split cuts.

Range: -1 to INT_MAX (default: 10)

-zeroHalfRowMaxFractionalCount

Skip ZeroHalf rows whose fractional count exceeds this threshold

If nonnegative, ZeroHalf skips any candidate row whose number of fractional variables in the current LP solution exceeds this threshold. Negative values disable the filter.

Range: -1 to INT_MAX (default: -1)

-zeroHalfRowMaxPairCount

Skip ZeroHalf rows whose pair count exceeds this threshold

If nonnegative, ZeroHalf skips any candidate row whose weakening pair count exceeds this threshold. Negative values disable the filter.

Range: -1 to INT_MAX (default: 150000)

`-zeroHalfSparseThreshold`

Active-node threshold for sparse ZeroHalf separation graph

If positive, ZeroHalf will use the sparse separation-graph implementation when the number of active separator nodes exceeds this threshold. A value of 0 forces sparse mode for testing. Negative values disable threshold-based switching, but sparse mode is still used automatically when the dense graph would be unsafe.

Range: -1 to INT_MAX (default: 8000)

`-passCuts`

Number of cut passes at root node

The default is 100 passes if less than 500 columns, 100 passes (but stop if the drop is small) if less than 5000 columns, 20 otherwise.

Range: −INT_MAX to INT_MAX (default: 100)

6 Heuristics

6.1 Constructive Heuristics

These heuristics do **not** require an existing feasible solution. They attempt to construct a feasible solution from scratch.

`-DivingCoefficient`

Whether to try Coefficient diving heuristic

Coefficient diving selects the fractional variable with the fewest constraint locks in the rounding direction. It rounds toward the direction with fewer locks (constraints that would be violated), breaking ties by smallest fractionality. This tends to minimize constraint violations during the dive. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `on`)

`-DivingFractional`

Whether to try Fractional diving heuristic

Fractional diving selects the fractional variable closest to an integer value and rounds it to the nearest integer. This is the simplest diving strategy: it always fixes the 'easiest' variable (smallest fractionality), minimizing the perturbation to the LP relaxation at each step. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `off`)

`-DivingGuided`

Whether to try Guided diving heuristic

Guided diving uses the best known feasible solution (incumbent) to decide the rounding direction: each fractional variable is rounded toward its value in the incumbent. Among candidates, it picks the variable with the smallest fractional distance in that direction. This explores the neighborhood of the incumbent, looking for improving solutions nearby. Requires at least one feasible solution. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `off`)

`-DivingLineSearch`

Whether to try Linesearch diving heuristic

Linesearch diving selects the variable where rounding to integrality requires the smallest step relative to how far the variable has moved from the root LP relaxation. It computes a ratio: (fractional gap to round) / (distance moved from root). A small ratio means the variable is nearly integer relative to its movement, making it a natural candidate to fix. The rounding direction follows the direction of movement from the root LP solution. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `off`)

`-DivingPseudocost`

Whether to try Pseudocost diving heuristic

Pseudocost diving uses estimated costs of rounding (pseudocosts) to select the variable and direction that maximizes a score balancing the fractionality and the ratio of pseudocosts. It rounds in the direction suggested by the root LP movement and pseudocost comparison, then scores each variable by fraction * (pCostDown+1)/(pCostUp+1) (or the reverse). This combines information from the LP relaxation trajectory with branching history to make informed rounding decisions. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `off`)

`-DivingSome`

Whether to try Diving heuristics

This switches on a random diving heuristic at various times. One may prefer to individually turn diving heuristics on or off. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `off`)

`-DivingVectorLength`

Whether to try Vectorlength diving heuristic

Vector length diving selects the variable that minimizes the ratio of objective degradation to the number of constraints the variable appears in (its column length). The rounding direction is

chosen to improve the objective. This favors variables that are 'well-connected' in the constraint matrix, since fixing a variable appearing in many constraints propagates more information to the LP. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `off`)

`-feasibilityPump`

Whether to try Feasibility Pump

This switches on feasibility pump heuristic at root. This is due to Fischetti and Lodi and uses a sequence of LPs to try and get an integer feasible solution. Some fine tuning is available by passFeasibilityPump.Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `on`)

`-greedyHeuristic`

Whether to use a greedy heuristic

Switches on a pair of greedy heuristic which will try and obtain a solution. It may just fix a percentage of variables and then try a small branch and cut run.Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `on`)

`-naiveHeuristics`

Whether to try some stupid heuristic

This is naive heuristics which, e.g., fix all integers with costs to zero!. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `off`)

`-pivotAndFix`

Whether to try Pivot and Fix heuristic

Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `off`)

`-randomizedRounding`

Whether to try randomized rounding heuristic

Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `off`)

`-Rens`

Whether to try Relaxation Enforced Neighborhood Search

Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve. Value 'on' just does 50 nodes. 200, 1000, and 10000 does that many nodes.

Values: `off`, `on`, `both`, `before`, `200`, `1000`, `10000`, `dj`, `djbefore`, `usesolution` (default: `off`)

`-roundingHeuristic`

Whether to use Rounding heuristic

This switches on a simple (but effective) rounding heuristic at each node of tree.

Values: `off`, `on`, `both`, `before` (default: `on`)

6.2 Improvement Heuristics

These heuristics require **at least one** existing feasible solution. They attempt to improve upon the incumbent.

`-Dins`

Whether to try Distance Induced Neighborhood Search

Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before`, `often` (default: `off`)

`-dwHeuristic`

Whether to try Dantzig Wolfe heuristic

This heuristic is very very compute intensive. It tries to find a Dantzig Wolfe structure and use that. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before`, `special`, `trial` (default: `off`)

`-localTreeSearch`

Whether to use local tree search

This switches on a local search algorithm when a solution is found. This is from Fischetti and Lodi and is not really a heuristic although it can be used as one. When used from this program it has limited functionality.

Values: `off`, `on`, `10`, `100`, `300` (default: `off`)

`-proximitySearch`

Whether to do proximity search heuristic

This heuristic looks for a solution close to the incumbent solution (Fischetti and Monaci, 2012). The idea is to define a sub-MIP without additional constraints but with a modified objective function intended to attract the search in the proximity of the incumbent. The approach works well for 0-1 MIPs whose solution landscape is not too irregular (meaning the there is reasonable probability of finding an improved solution by flipping a small number of binary variables), in particular when it is applied to the first heuristic solutions found at the root node. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before`, `10`, `100`, `300` (default: `off`)

`-Rins`

Whether to try Relaxed Induced Neighborhood Search

Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before`, `often` (default: `on`)

`-VndVariableNeighborhoodSearch`

Whether to try Variable Neighborhood Search

Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before`, `intree` (default: `off`)

6.3 Improvement Heuristics (2+ solutions)

These heuristics require **at least two** existing feasible solutions. They combine or crossover multiple solutions.

`-combineSolutions`

Whether to use combine solution heuristic

This switches on a heuristic which does branch and cut on the problem given by just using variables which have appeared in one or more solutions. It is obviously only tried after two or more solutions.Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before`, `onequick`, `bothquick`, `beforequick` (default: `off`)

`-combine2Solutions`

Whether to use crossover solution heuristic

This heuristic does branch and cut on the problem given by fixing variables which have the same value in two or more solutions. It obviously only tries after two or more solutions. Value 'on' means to use the heuristic in each node of the tree, i.e. after preprocessing. Value 'before' means use the heuristic only if option doHeuristics is used. Value 'both' means to use the heuristic if option doHeuristics is used and during solve.

Values: `off`, `on`, `both`, `before` (default: `off`)

6.4 General Heuristic Settings

`-heuristicsOnOff`

Switches most heuristics on or off

This can be used to switch on or off all heuristics. Then you can set individual ones off or on. CbcTreeLocal is not included as it dramatically alters search.

Values: `off`, `on`, `both`, `before` (default: `on`)

`-doHeuristic`

Do heuristics before any preprocessing

Normally heuristics are done in branch and bound. It may be useful to do them outside. Only those heuristics with 'both' or 'before' set will run. Doing this may also set cutoff, which can help with preprocessing.

Values: `off`, `on` (default: `off`)

`-forceSolution`

Whether to use given solution as crash for BAB

If on then tries to branch to solution given by AMPL or priorities file.

Values: `off`, `on` (default: `off`)

`-depthMiniBab`

Depth at which to try mini branch-and-bound

Rather a complicated parameter but can be useful. -1 means off for large problems but on as if -12 for problems where rows+columns<500, -2 means use Cplex if it is linked in. Otherwise if negative then go into depth first complete search fast branch and bound when depth>= -value-2 (so -3 will use this at depth>=1). This mode is only switched on after 500 nodes. If you really want to switch it off for small problems then set this to -999. If >=0 the value doesn't matter very much. The code will do approximately 100 nodes of fast branch and bound every now and then at depth>=5. The actual logic is too twisted to describe here. The default has been changed from -1 to +1. This uses Clp and saves factorizations etc to be faster.

Range: −INT_MAX to INT_MAX (default: 1)

-diveOpt

Diving options

If >2 && <=8 then modify diving options - 3 only at root and if no solution, 4 only at root and if this heuristic has not got solution, 5 decay only if no solution, 6 if depth <3 or decay, 7 run up to 2 times if solution found 4 otherwise, 8 fire at every node until first incumbent then revert to default d^2/2^d schedule (aggressive feasibility mode), >10 All only at root (DivingC normal as value-10), >20 All with value-20).

Range: -1 to 20 (default: 2)

-diveSolves

Diving solve option

If >0 then do up to this many solves. However, the last digit is ignored and used for extra options: 1-3 enables fixing of satisfied integer variables (but not at bound), where 1 switches this off for that dive if the dive goes infeasible, and 2 switches it off permanently if the dive goes infeasible.

Range: -1 to 200000 (default: 100)

-passFeasibilityPump

How many passes in feasibility pump

This fine tunes the Feasibility Pump heuristic by doing more or fewer passes.

Range: 0 to 10000 (default: 30)

-pumpTune

Dubious ideas for feasibility pump

This fine tunes Feasibility Pump >=10000000 use as objective weight switch >=1000000 use as accumulate switch >=1000 use index+1 as number of large loops ==100 use objvalue +0.05*fabs(objvalue) as cutoff OR fakeCutoff if set %100 == 10,20 affects how each solve is done 1 == fix ints at bounds, 2 fix all integral ints, 3 and continuous at bounds. If accumulate is on then after a major pass, variables which have not moved are fixed and a small branch and bound is tried.

Range: 0 to 1000000000 (default: 1005043)

-hOptions

Heuristic options

Value 1 stops heuristics immediately if the allowable gap has been reached. Other values are for the feasibility pump - 2 says do exact number of passes given, 4 only applies if an initial cutoff has been given and says relax after 50 passes, while 8 will adapt the cutoff rhs after the first solution if it looks as if the code is stalling.

Range: −INT_MAX to INT_MAX (default: 0)

-fpumpPassFreq

Print feasibility pump progress every N passes (0 = disabled).

Range: 0 to 1000000 (default: 0)

`-artificialCost`

Costs >= this treated as artificials in feasibility pump

Range: 0 to inf (default: 0)

`-rinsCloseMaxDist`

Maximum fractional distance for RINS close-fixing fallback

When the standard RINS fix-count threshold (>20%% of integers must agree between LP and best solution) is not met, integer variables whose current LP value is within this distance of the corresponding best-solution integer value are sorted by closeness and greedily fixed (closest first) until the threshold is satisfied. A value of 0.0 disables the fallback. Default: 0.4. Typical useful values: 0.2-0.5.

Range: 0 to 0.5 (default: 0.4)

7 Branching

`-branchPriorities`

What rule (if any) to use in prioritizing variables for branching

What rule (if any) to use in prioritizing variables for branching - 'priorities' assigns highest priority to variables with largest absolute cost. This primitive strategy can be surprisingly effective. - 'columnorder' assigns the priorities with respect to the column ordering. - '01first' ('01last') gives highest priority to binary variables. - 'length' assigns high priority to variables that occur in many constraints.

Values: `off`, `pri!orities`, `column!Order`, `01f!irst?`, `01l!ast?`, `length!?`, `singletons`, `nonzero`, `general!Force?` (default: `off`)

`-nodeStrategy`

What strategy to use to select the next node from the branch and cut tree

Normally before a feasible solution is found, CBC will choose a node with fewest infeasibilities. Alternatively, one may choose tree-depth as the criterion. This requires the minimal amount of memory, but may take a long time to find the best solution. Additionally, one may specify whether up or down branches must be selected first (the up-down choice will carry on after a first solution has been bound). The choice 'hybrid' does breadth first on small depth nodes and then switches to 'fewest'.

Values: `hybrid`, `fewest`, `depth`, `upfewest`, `downfewest`, `updepth`, `downdepth` (default: `fewest`)

`-OrbitalBranching`

Whether to try orbital branching

This switches on Orbital branching. Value 'on' just adds orbital, 'strong' tries extra fixing in strong branching.'cuts' just adds global cuts to break symmetry.'lightweight' is as on where computation seems cheap

Values: `off`, `slowish`, `strong`, `force`, `simple`, `on`, `lightweight`, `moreprinting`, `cuts`, `cutslight` (default: `off`)

-sosPrioritize

How to deal with SOS priorities

This sets priorities for SOS. Values 'high' and 'low' just set a priority relative to the for integer variables. Value 'orderhigh' gives first highest priority to the first SOS and integer variables a low priority. Value 'orderlow' gives integer variables a high priority then SOS in order.

Values: `off`, `high`, `low`, `orderhigh`, `orderlow` (default: `off`)

-trustPseudocosts

Number of branches before we trust pseudocosts

Using strong branching computes pseudo-costs. After this many times for a variable we just trust the pseudo costs and do not do any more strong branching.

Range: -3 to INT_MAX (default: 10)

-strongBranching

Number of variables to look at in strong branching

In order to decide which variable to branch on, the code will choose up to this number of unsatisfied variables and try mini up and down branches. The most effective one is chosen. If a variable is branched on many times then the previous average up and down costs may be used - see number before trust.

Range: 0 to 999999 (default: 5)

8 Tolerances

-increment

A new solution must be at least this much better than the incumbent

Whenever a solution is found the bound on future solutions is set to the objective of the solution (in a minimization sense) plus the specified increment. If this option is not specified, the code will try and work out an increment. E.g., if all objective coefficients are multiples of 0.01 and only integer variables have entries in objective then the increment can be set to 0.01. Be careful if you set this negative!

Range: $-\infty$ to ∞ (default: 0.0001)

-infeasibilityWeight

Each integer infeasibility is expected to cost this much

A primitive way of deciding which node to explore next. Satisfying each integer infeasibility is expected to cost this much.

Range: 0 to ∞ (default: 0)

-integerTolerance

For an optimal solution, no integer variable may be farther than this from an integer value

When checking a solution for feasibility, if the difference between the value of a variable and the nearest integer is less than the integer tolerance, the value is considered to be integral. Beware of setting this smaller than the primal tolerance.

Range: 1e-20 to 0.5 (default: 1e-06)

`-dualTolerance`

For an optimal solution no dual infeasibility may exceed this value

Normally the default tolerance is fine, but one may want to increase it a bit if the dual simplex algorithm seems to be having a hard time. One method which can be faster is to use a large tolerance e.g. 1.0e-4 and the dual simplex algorithm and then to clean up the problem using the primal simplex algorithm with the correct tolerance (remembering to switch off presolve for this final short clean up phase).

Range: 1e-20 to inf (default: 1e-06)

`-primalTolerance`

For a feasible solution no primal infeasibility, i.e., constraint violation, may exceed this value

Normally the default tolerance is fine, but one may want to increase it a bit if the primal simplex algorithm seems to be having a hard time.

Range: 1e-20 to inf (default: 1e-06)

`-zeroTolerance`

Kill all coefficients whose absolute value is less than this value

This applies to reading mps files (and also lp files if KILL_ZERO_READLP defined)

Range: 1e-100 to 1e-05 (default: 1e-20)

9 Conflict Graph

`-cgraph`

Whether to use the conflict graph-based preprocessing and cut separation routines.

This switches the conflict graph-based preprocessing and cut separation routines (CglBKClique, CglOddWheel and CliqueStrengthening) on or off. Values: off: turns these routines off; on: turns these routines on; clq: turns these routines off and enables the cut separator of CglClique.

Values: `off`, `on`, `clq` (default: `on`)

`-bkpivoting`

Pivoting strategy used in Bron-Kerbosch algorithm

Range: 0 to 6 (default: 3)

`-bkmaxcalls`

Maximum number of recursive calls made by Bron-Kerbosch algorithm

Range: 1 to INT_MAX (default: 1000)

-bkclqextmethod

Strategy used to extend violated cliques found by BK Clique Cut Separation routine

Sets the method used in the extension module of BK Clique Cut Separation routine: 0=no extension; 1=random; 2=degree; 3=modified degree; 4=reduced cost(inversely proportional); 5=reduced cost(inversely proportional) + modified degree

Range: 0 to 5 (default: 4)

-oddwextmethod

Strategy used to search for wheel centers for the cuts found by Odd Wheel Cut Separation routine

Sets the method used in the extension module of Odd Wheel Cut Separation routine: 0=no extension; 1=one variable; 2=clique

Range: 0 to 2 (default: 2)

10 Strategy

-strategy

Switches on groups of features

Selects a preset configuration that adjusts cuts, heuristics, and solver tuning as a group.

easy (0): A lighter configuration. Uses Gomory cuts with a looser tolerance (0.01 at root), shorter FPump runs (20 passes, tune=1003), no preprocessing tuning, and disables RINS and DivingCoefficient.

default (1): The recommended configuration. Tightens Gomory and TwoMir cut tolerances, runs FPump more aggressively (30 passes, tune=1005043), enables DivingCoefficient and RINS heuristics, and activates probing cuts (ifmove). This is what runs when no -strategy flag is given.

aggressive (2): Reserved for future use; currently identical to default.

Values: `easy`, `default`, `aggressive` (default: `default`)

-experiment

Whether to use testing features

Defines how adventurous you want to be in using new ideas. 0 then no new ideas, 1 fairly sensible, 2 a bit dubious, 3 you are on your own!

Range: -1 to 200000 (default: 0)

-hotStartMaxIts

Maximum iterations on hot start

Range: 0 to INT_MAX (default: 100)

-multipleRootPasses

Do multiple root passes to collect cuts and solutions

Solve (in parallel, if enabled) the root phase this number of times, each with its own different seed, and collect all solutions and cuts generated. The actual format is aabbcc where aa is the number of extra passes; if bb is non zero, then it is number of threads to use (otherwise uses threads setting); and cc is the number of times to do root phase. The solvers do not interact with each other. However if extra passes are specified then cuts are collected and used in later passes - so there is interaction there. Some parts of this implementation have their origin in idea of Andrea Lodi, Matteo Fischetti, Michele Monaci, Domenico Salvagnin, and Andrea Tramontani.

Range: 0 to INT_MAX (default: 0)

`-options`

Fine tuning of specialOptions

If set Or's with specialOptions just before entering branchAndBound.

Range: 0 to INT_MAX (default: 0)

`-pumpCutoff`

Fake cutoff for use in feasibility pump

A value of 0.0 means off. Otherwise, add a constraint forcing objective below this value in feasibility pump

Range: -inf to inf (default: 0)

`-pumpIncrement`

Fake increment for use in feasibility pump

A value of 0.0 means off. Otherwise, add a constraint forcing objective below this value in feasibility pump

Range: -inf to inf (default: 0)

`-fractionforBAB`

Fraction in feasibility pump

After a pass in the feasibility pump, variables which have not moved about are fixed and if the preprocessed model is smaller than this fraction of the original problem, a few nodes of branch and bound are done on the reduced problem.

Range: 1e-05 to 1.1 (default: 0.5)

`-fakeBound`

All bounds <= this value - DEBUG

Range: 1 to 1000000000000000.0 (default: 0)

11 Solving

`-solve`

invoke branch and cut to solve the current problem

This does branch and cut. There are many parameters which can affect the performance. First just try with default cbcSettings and look carefully at the log file. Did cuts help? Did they take too long? Look at output to see which cuts were effective and then do some tuning. You will see that the options for cuts are off, on, root and ifmove. Off is obvious, on means that this cut generator will be tried in the branch and cut tree (you can fine tune using 'depth'). Root means just at the root node while 'ifmove' means that cuts will be used in the tree if they look as if they are doing some good and moving the objective value. If pre-processing reduced the size of the problem or strengthened many coefficients then it is probably wise to leave it on. Switch off heuristics which did not provide solutions. The other major area to look at is the search. Hopefully good solutions were obtained fairly early in the search so the important point is to select the best variable to branch on. See whether strong branching did a good job - or did it just take a lot of iterations. Adjust the strongBranching and trustPseudoCosts parameters.

`-initialSolve`

Solve to continuous optimum

This just solves the problem to the continuous optimum, without adding any cuts.

`-strengthen`

Create strengthened problem

This creates a new problem by applying the root node cuts. All tight constraints will be in resulting problem.

`-constraintfromCutoff`

Whether to use cutoff as constraint

For some problems, cut generators and general branching work better if the problem would be infeasible if the cost is too high. If this option is enabled, the objective function is added as a constraint which right hand side is set to the current cutoff value (objective value of best known solution)

Values: `off`, `on`, `variable`, `forcevariable`, `conflict` (default: `off`)

`-lpMethod`

Which LP algorithm to use for the initial LP relaxation solve

Controls which LP algorithm is used when -solve or -initialSolve triggers the root LP relaxation. dual: dual simplex (default). primal: primal simplex. barrier: interior-point (barrier) method. auto: ML-based per-instance recommendation, using a classifier trained on instance features (see CbcLpParamScorer) to pick the LP method/perturbation/scaling settings expected to solve fastest.

Values: `dual`, `primal`, `barrier`, `auto` (default: `dual`)

`-maxSavedSolutions`

Maximum number of solutions to save

Number of solutions to save.

Range: 0 to INT_MAX (default: 10)

`-randomCbcSeed`

Random seed for Cbc

Allows initialization of the random seed for pseudo-random numbers used in heuristics such as the Feasibility Pump to decide whether to round up or down. The special value of 0 lets Cbc use the time of the day for the initial seed.

Range: -1 to INT_MAX (default: 42)

`-direction`

Minimize or maximize

The default is minimize - use 'direction maximize' for maximization. You can also use the parameters_ 'maximize' or 'minimize'.

Values: `max!imize`, `min!imize`, `zero` (default: `min(imize)`)

`-maximize`

Set optimization direction to maximize

The default is minimize - use 'maximize' for maximization. A synonym for 'direction maximize'.

`-minimize`

Set optimization direction to minimize

The default is minimize - use 'maximize' for maximization. This should only be necessary if you have previously set maximization. A synonym for 'direction minimize'.

`-allSlack`

Set basis back to all slack and reset solution

Mainly useful for tuning purposes. Normally the first dual or primal will be using an all slack basis anyway.

`-barrier`

Solve using primal dual predictor corrector algorithm

This command solves the current model using the primal dual predictor corrector algorithm. You may want to link in an alternative ordering and factorization. It will also solve models with quadratic objectives.

`-dualSimplex`

Do dual simplex algorithm

This command solves the continuous relaxation of the current model using the dual steepest edge algorithm. The time and iterations may be affected by settings such as presolve, scaling, crash and also by dual pivot method, fake bound on variables and dual and primal tolerances.

-eitherSimplex

Do dual or primal simplex algorithm

This command solves the continuous relaxation of the current model using the dual or primal algorithm, based on a dubious analysis of model.

-guess

Guesses at good parameters

This looks at model statistics and does an initial solve setting some parameters which may help you to think of possibilities.

-network

Tries to make network matrix

Clp will go faster if the matrix can be converted to a network. The matrix operations may be a bit faster with more efficient storage, but the main advantage comes from using a network factorization. It will probably not be as fast as a specialized network code.

-parametrics

Import data from file and do parametrics

This will read a file with parametric data from the given file name and then do parametrics. It will use the default directory given by 'directory'. A name of '$' will use the previous value for the name. This is initialized to '', i.e. it must be set. This can not read from compressed files. File is in modified csv format - a line ROWS will be followed by rows data while a line COLUMNS will be followed by column data. The last line should be ENDATA. The ROWS line must exist and is in the format ROWS, inital theta, final theta, interval theta, n where n is 0 to get CLPI0062 message at interval or at each change of theta and 1 to get CLPI0063 message at each iteration. If interval theta is 0.0 or >= final theta then no interval reporting. n may be missed out when it is taken as 0. If there is Row data then there is a headings line with allowed headings - name, number, lower(rhs change), upper(rhs change), rhs(change). Either the lower and upper fields should be given or the rhs field. The optional COLUMNS line is followed by a headings line with allowed headings - name, number, objective(change), lower(change), upper(change). Exactly one of name and number must be given for either section and missing ones have value 0.0.

-plusMinus

Tries to make +- 1 matrix

Clp will go slightly faster if the matrix can be converted so that the elements are not stored and are known to be unit. The main advantage is memory use. Clp may automatically see if it can convert the problem so you should not need to use this.

-primalSimplex

Do primal simplex algorithm

This command solves the continuous relaxation of the current model using the primal algorithm. The default is to use exact devex. The time and iterations may be affected by settings such as presolve, scaling, crash and also by column selection method, infeasibility weight and dual and primal tolerances.

`-reallyScale`

Scales model in place

`-reverse`

Reverses sign of objective

Useful for testing if maximization works correctly

`-direction`

Minimize or Maximize

The default is minimize - use 'direction maximize' for maximization. You can also use the parameters 'maximize' or 'minimize'.

Values: `min!imize`, `max!imize`, `zero` (default: `min(imize)`)

`-vector`

Try and use vector instructions in simplex

At present only for Intel architectures - but could be extended. Uses avx2 or avx512 instructions. Uses different storage for matrix - can be of benefit without instruction set on some problems. I may add pool to switch on a pool matrix

Values: `off`, `on`, `ones` (default: `off`)

`-decompose`

Whether to try decomposition

0 - off, 1 choose blocks >1 use as blocks Dantzig Wolfe if primal, Benders if dual - uses sprint pass for number of passes

Range: −INT_MAX to INT_MAX (default: 0)

`-dualize`

Solves dual reformulation

Don't even think about it.

Range: 0 to 4 (default: 3)

`-randomSeed`

Random seed for Clp

Initialization of the random seed for pseudo-random numbers used to break ties in degenerate problems. This may yield a different continuous optimum and, in the context of Cbc, different cuts and heuristic solutions. The special value of 0 lets CLP use the time of the day for the initial seed.

Range: 0 to INT_MAX (default: 1234567)

12 Simplex

-KKT

Whether to use KKT factorization in barrier

Values: off, on (default: off)

-perturbation

Whether to perturb the problem

Perturbation helps to stop cycling, but CLP uses other measures for this. However, large problems and especially ones with unit elements and unit right hand sides or costs benefit from perturbation. Normally CLP tries to be intelligent, but one can switch this off.

Values: off, on (default: on)

-crash

Whether to create basis for problem

If crash is set to 'on' and there is an all slack basis then Clp will flip or put structural variables into the basis with the aim of getting dual feasible. On average, dual simplex seems to perform better without it and there are alternative types of 'crash' for primal simplex, e.g. 'idiot' or 'sprint'. A variant due to Solow and Halim which is as 'on' but just flips is also available.

Values: off, on, so!low_halim, lots, free, zero, single!ton, idiot1, idiot2, idiot3, idiot4, idiot5, idiot6, idiot7 (default: off)

-dualPivot

Dual pivot choice algorithm

The Dantzig method is simple but its use is deprecated. Steepest is the method of choice and there are two variants which keep all weights updated but only scan a subset each iteration. Partial switches this on while automatic decides at each iteration based on information about the factorization. The PE variants add the Positive Edge criterion. This selects incoming variables to try to avoid degenerate moves. See also option psi.

Values: auto!matic, dant!zig, partial, steep!est, PEsteep!est, PEdantzig (default: auto(matic))

-factorization

Which factorization to use

The default is to use the normal CoinFactorization, but other choices are a dense one, OSL's, or one designed for small problems.

Values: normal, dense, simple, osl (default: normal)

-primalPivot

Primal pivot choice algorithm

The Dantzig method is simple but its use is deprecated. Exact devex is the method of choice and there are two variants which keep all weights updated but only scan a subset each iteration. Partial switches this on while 'change' initially does 'dantzig' until the factorization becomes denser. This is still a work in progress. The PE variants add the Positive Edge criterion. This

selects incoming variables to try to avoid degenerate moves. See also Towhidi, M., Desrosiers, J., Soumis, F., The positive edge criterion within COIN-OR's CLP; Omer, J., Towhidi, M., Soumis, F., The positive edge pricing rule for the dual simplex.

Values: `auto!matic`, `exa!ct`, `dant!zig`, `part!ial`, `steep!est`, `change`, `sprint`, `PEsteep!est`, `PEdantzig` (default: `auto(matic)`)

`-denseThreshold`

Threshold for using dense factorization

If processed problem <= this use dense factorization

Range: -1 to 10000 (default: -1)

`-idiotCrash`

Whether to try idiot crash

This is a type of 'crash' which works well on some homogeneous problems. It works best on problems with unit elements and rhs but will do something to any model. It should only be used before the primal simplex algorithm. It can be set to -1 when the code decides for itself whether to use it, 0 to switch off, or n > 0 to do n passes.

Range: -1 to INT_MAX (default: -1)

`-maxFactor`

Maximum number of iterations between refactorizations

If this is left at its default value of 200 then CLP will guess a value to use. CLP may decide to re-factorize earlier for accuracy.

Range: 1 to INT_MAX (default: 200)

`-maxIterations`

Maximum number of iterations before stopping

This can be used for testing purposes. The corresponding library call setMaximumIterations(value) can be useful. If the code stops on seconds or by an interrupt this will be treated as stopping on maximum iterations. This is ignored in branchAndCut - use maxN!odes.

Range: 0 to INT_MAX (default: 2147483647)

`-moreSpecialOptions`

Yet more dubious options for Simplex

See ClpSimplex.hpp.

Range: 0 to INT_MAX (default: 0)

`-pertValue`

Method of perturbation

Range: -5000 to 102 (default: 50)

-sprintCrash

Whether to try sprint crash

For long and thin problems this method may solve a series of small problems created by taking a subset of the columns. The idea as 'Sprint' was introduced by J. Forrest after an LP code of that name of the 60's which tried the same tactic (not totally successfully). CPLEX calls it 'sifting'. -1 lets CLP automatically choose the number of passes, 0 is off, n is number of passes

Range: -1 to INT_MAX (default: -1)

-slpValue

Number of slp passes before primal

If you are solving a quadratic problem using primal then it may be helpful to do some sequential Lps to get a good approximate solution.

Range: -50000 to 50000 (default: 0)

-smallFactorization

Threshold for using small factorization

If processed problem <= this use small factorization

Range: -1 to 10000 (default: -1)

-specialOptions

Dubious options for Simplex - see ClpSimplex.hpp

Range: 0 to INT_MAX (default: 0)

-dualBound

Initially algorithm acts as if no gap between bounds exceeds this value

The dual algorithm in Clp is a single phase algorithm as opposed to a two phase algorithm where you first get feasible then optimal. If a problem has both upper and lower bounds then it is trivial to get dual feasible by setting non basic variables to correct bound. If the gap between the upper and lower bounds of a variable is more than the value of dualBound Clp introduces fake bounds so that it can make the problem dual feasible. This has the same effect as a composite objective function in the primal algorithm. Too high a value may mean more iterations, while too low a bound means the code may go all the way and then have to increase the bounds. OSL had a heuristic to adjust bounds, maybe we need that here.

Range: 1e-20 to ∞ (default: 10000000000)

-primalWeight

Initially algorithm acts as if it costs this much to be infeasible

The primal algorithm in Clp is a single phase algorithm as opposed to a two phase algorithm where you first get feasible then optimal. So Clp is minimizing this weight times the sum of primal infeasibilities plus the true objective function (in minimization sense). Too high a value may mean more iterations, while too low a value means the algorithm may iterate into the wrong directory for long and then has to increase the weight in order to get feasible.

Range: 1e-20 to inf (default: 10000000000)

-psi

Two-dimension pricing factor for Positive Edge criterion

The Positive Edge criterion has been added to select incoming variables to try and avoid degenerate moves. Variables not in the promising set have their infeasibility weight multiplied by psi, so 0.01 would mean that if there were any promising variables, then they would always be chosen, while 1.0 effectively switches the algorithm off. There are two ways of switching this feature on. One way is to set psi to a positive value and then the Positive Edge criterion will be used for both primal and dual simplex. The other way is to select PEsteepest in dualpivot choice (for example), then the absolute value of psi is used. Code donated by Jeremy Omer. See Towhidi, M., Desrosiers, J., Soumis, F., The positive edge criterion within COIN-OR's CLP; Omer, J., Towhidi, M., Soumis, F., The positive edge pricing rule for the dual simplex.

Range: -1.1 to 1.1 (default: -0.5)

13 Barrier

-cholesky

Which cholesky algorithm

For a barrier code to be effective it needs a good Cholesky ordering and factorization. The native ordering and factorization is not state of the art, although acceptable. You may want to link in one from another source. See Makefile.locations for some possibilities.

Values: `native`, `dense`, `fudge!Long_dummy`, `wssmp_dummy`, `Uni!versityOfFlorida`, `Taucs_dummy`, `Mumps_dummy`, `Pardiso_dummy` (default: `native`)

-crossover

Whether to get a basic solution with the simplex algorithm after the barrier algorithm finished

Interior point algorithms do not obtain a basic solution. This option will crossover to a basic solution suitable for ranging or branch and cut. With the current state of the solver for quadratic programs it may be a good idea to switch off crossover for this case (and maybe presolve as well) - the option 'maybe' does this.

Values: `off`, `on`, `maybe`, `presolve` (default: `on`)

-gamma(Delta)

Whether to regularize barrier

Values: `off`, `on`, `gamma`, `delta`, `onstrong`, `gammastrong`, `deltastrong` (default: `off`)

14 Scaling

-scaling

Whether to scale problem

Scaling can help in solving problems which might otherwise fail because of lack of accuracy. It can also reduce the number of iterations. It is not applied if the range of elements is small. When the solution is evaluated in the unscaled problem, it is possible that small primal and/or

dual infeasibilities occur. - 'equilibrium' uses the largest element for scaling. - 'geometric' uses the squareroot of the product of largest and smallest element. - 'auto' lets CLP choose a method that gives the best ratio of the largest element to the smallest one.

Values: `off`, `equi!librium`, `geo!metric`, `auto!matic`, `dynamic`, `rows!only` (default: `auto(matic)`)

`-objectiveScale`

Scale factor to apply to objective

If the objective function has some very large values, you may wish to scale them internally by this amount. It can also be set by autoscale. It is applied after scaling. You are unlikely to need this.

Range: -inf to inf (default: 1)

`-reallyObjectiveScale`

Scale factor to apply to objective in place

You can set this to -1.0 to test maximization or other to stress code

Range: -inf to inf (default: 1)

`-rhsScale`

Scale factor to apply to rhs and bounds

If the rhs or bounds have some very large meaningful values, you may wish to scale them internally by this amount. It can also be set by autoscale. This should not be needed.

Range: -inf to inf (default: 1)

15 Output

`-statistics`

Print some statistics

This command prints some statistics for the current model. If log level >1 then more is printed. These are for presolved model if presolve on (and unscaled).

`-printMask`

Control printing of solution with a regular expression

If set then only those names which match mask are printed in a solution. '?' matches any character and '*' matches any set of characters. The default is '' (unset) so all variables are printed. This is only active if model has names.

`-precisionOutput`

Handle format precision with string print mask

Precision: %.nf -> n digits after decimal; %.ng -> n significant digits; Width: %mw -> minimum field width, padded with spaces by default. Remember the f or g at end as %18.5 by itself gives garbage.

-messages

Controls whether standardised message prefix is printed

By default, messages have a standard prefix, such as: Cbc0005 2261 Objective 109.024 Primal infeas 944413 (758) but this program turns this off to make it look more friendly. It can be useful to turn them back on if you want to be able to 'grep' for particular messages or if you intend to override the behavior of a particular message.

Values: `off`, `on` (default: `off`)

-checktimeFrequency

How often to check time for stopping

Checking the time costs more than one might think. In cbc one does not normally need to stop after generating a cut or doing an iteration. So less checks less often and often is more likely to check every iteration.

Values: `less`, `often` (default: `often`)

-printingOptions

Print options

This changes the amount and format of printing a solution: normal - nonzero column variables integer - nonzero integer column variables special - in format suitable for OsiRowCutDebugger rows - nonzero column variables and row activities all - all column variables and row activities.

For non-integer problems 'integer' and 'special' act like 'normal'. Also see printMask for controlling output.

Values: `normal`, `integer`, `special`, `rows`, `all`, `csv`, `bound!ranging`, `rhs!ranging`, `objective!ranging`, `stats`, `boundsint`, `boundsall`, `fixint`, `fixall`, `allcsv` (default: `normal`)

-timeMode

Whether to use CPU or elapsed time

elapsed uses elapsed (wall-clock) time for stopping, while cpu uses CPU time. Elapsed is the default as it is more natural for users. (On Windows, elapsed time is always used).

Values: `cpu`, `elapsed` (default: `elapsed`)

-logLevel

Level of detail in CBC output.

If set to 0 then there should be no output in normal circumstances. A value of 1 is probably the best value for most uses, while 2 and 3 give more information.

Range: -1 to 999999 (default: 1)

-lplogLevel

Level of detail in LP solver output.

If set to 0 then there should be no output in normal circumstances. A value of 1 is probably the best value for most uses, while 2 and 3 give more information.

Range: -1 to 999999 (default: 1)

-flushPerNewLine

Flush output after every message line.

When set to 1 (default), each output line is flushed immediately. This is already the default behaviour of CoinMessageHandler (which calls fflush after every CoinMessageEol). Setting to 0 is reserved for future use to allow batching of output for performance in non-interactive scenarios.

Range: 0 to 1 (default: 0)

-useUTF8

Use UTF-8 characters in output.

Controls whether UTF-8 characters ($\in$, κ, —) are used in solver output. -1 (default) auto-detects from the locale (LANG/LC_ALL environment variables). 0 forces ASCII-only output. 1 forces UTF-8 output.

Range: -1 to 1 (default: 0)

-compactTables

Use compact (borderless) table style in output.

Controls the table style used for progress tables (LP relaxation, preprocessing, feasibility pump, cut generation, branch-and-bound). 1 (default) uses a compact style: no column borders, columns separated by spaces, and a single thin rule under the header (a continuous line in UTF-8 mode, per-column dashes in ASCII mode). 0 uses the full bordered box-drawing style.

Range: 0 to 1 (default: 0)

-lpIterFreq

Print LP progress every N iterations (0 = disabled).

When solving the LP relaxation at the root node, print a progress row every N iterations. Set to 0 to disable iteration-based printing. Use lpTimeFreq for time-based printing.

Range: 0 to INT_MAX (default: 0)

-outputFormat

Which output format to use

Normally export will be done using normal representation for numbers and two values per line. You may want to do just one per line (for grep or suchlike) and you may wish to save with absolute accuracy using a coded version of the IEEE value. A value of 2 is normal. Otherwise, odd values give one value per line, even values two. Values of 1 and 2 give normal format, 3 and 4 give greater precision, 5 and 6 give IEEE values. When exporting a basis, 1 does not save values, 2 saves values, 3 saves with greater accuracy and 4 saves in IEEE format.

Range: 1 to 6 (default: 2)

-pOptions

Dubious print options

If this is greater than 0 then presolve will give more information and branch and cut will give statistics

Range: 0 to INT_MAX (default: 0)

-lpTimeFreq

Print LP progress every N seconds (0 = disabled).

When solving the LP relaxation at the root node, print a progress row every N seconds. Set to 0 to disable time-based printing. Use lpIterFreq for iteration-based printing.

Range: 0 to inf (default: 5)

-fpumpTimeFreq

Print feasibility pump progress every N seconds (0 = disabled, default 5).

Range: 0 to 10000000000 (default: 5)

-bufferedMode

Whether to flush print buffer

Default is on, off switches on unbuffered output

Values: `off`, `on` (default: `on`)

-messages

Controls if Clpnnnn is printed

The default behavior is to put out messages such as: Clp0005 2261 Objective 109.024 Primal infeas 944413 (758) but this program turns this off to make it look more friendly. It can be useful to turn them back on if you want to be able to 'grep' for particular messages or if you intend to override the behavior of a particular message.

Values: `off`, `on` (default: `off`)

-allCommands

What priority level of commands to print

For the sake of your sanity, only the more useful and simple commands are printed out on ?.

Values: `all`, `more`, `important` (default: `more`)

-printingOptions

Print options

This changes the amount and format of printing a solution: normal - nonzero column variables integer - nonzero integer column variables special - in format suitable for OsiRowCutDebugger rows - nonzero column variables and row activities all - all column variables and row activities.

For non-integer problems 'integer' and 'special' act like 'normal'. Also see printMask for controlling output.

Values: `normal`, `integer`, `special`, `rows`, `all`, `csv`, `bound!ranging`, `rhs!ranging`, `objective!ranging`, `stats`, `boundsint`, `boundsall`, `fixint`, `fixall` (default: `normal`)

`-cppGenerate`

Generates C++ code

Once you like what the stand-alone solver does then this allows you to generate user_driver.cpp which approximates the code. 0 gives simplest driver, 1 generates saves and restores, 2 generates saves and restores even for variables at default value. 4 bit in cbc generates size dependent code rather than computed values. This is now deprecated as you can call stand-alone solver - see Cbc/examples/driver4.cpp.

Range: -1 to 50000 (default: 0)

`-progressInterval`

Time interval for printing progress

This sets a minimum interval for some printing - elapsed seconds

Range: -inf to inf (default: 0.7)

16 I/O

`-export`

Export model as mps file

This will write an MPS format file to the given file name. It will use the default directory given by 'directory'. A name of '$' will use the previous value for the name. This is initialized to 'default.mps'. It can be useful to get rid of the original names and go over to using Rnnnnnnn and Cnnnnnnn. This can be done by setting 'keepnames' off before importing mps file.

`-import`

Import model from file

This will read an MPS format file from the given file name. It will use the default directory given by 'directory'. A name of '$' will use the previous value for the name. This is initialized to '', i.e., it must be set. If you have libgz then it can read compressed files 'xxxxxxxx.gz'.

`-printSolution`

writes solution to file (or stdout)

This will write a binary solution file to the file set by solFile.

`-mipStart`

reads an initial feasible solution from file

The MIPStart allows one to enter an initial integer feasible solution to CBC. Values of the main decision variables which are active (have non-zero values) in this solution are specified in a text file. The text file format used is the same of the solutions saved by CBC, but not all fields are required to be filled. First line may contain the solution status and will be ignored, remaining lines contain column indexes, names and values as in this example:

Stopped on iterations - objective value 57597.00000000 0 x(1,1,2,2) 1 1 x(3,1,3,2) 1 5 v(5,1) 2 33 x(8,1,5,2) 1 ...

Column indexes are also ignored since pre-processing can change them. There is no need to include values for continuous or integer auxiliary variables, since they can be computed based on main decision variables. Starting CBC with an integer feasible solution can dramatically improve its performance: several MIP heuristics (e.g. RINS) rely on having at least one feasible solution available and can start immediately if the user provides one. Feasibility Pump (FP) is a heuristic which tries to overcome the problem of taking too long to find feasible solution (or not finding at all), but it not always succeeds. If you provide one starting solution you will probably save some time by disabling FP.

Knowledge specific to your problem can be considered to write an external module to quickly produce an initial feasible solution - some alternatives are the implementation of simple greedy heuristics or the solution (by CBC for example) of a simpler model created just to find a feasible solution.

Silly options added. If filename ends .low then integers not mentioned are set low - also .high, .lowcheap, .highcheap, .lowexpensive, .highexpensive where .lowexpensive sets costed ones to make expensive others low. Also if filename starts empty. then no file is read at all - just actions done.

Question and suggestions regarding MIPStart can be directed to haroldo.santos@gmail.com.

`-readPriorities`

reads priorities from file

Read priorities from the file name designated by PRIORITYFILE. File is in csv format with allowed headings - name, number, priority, direction, up, down, solution. Exactly one of name and number must be given.

`-readModel`

Reads problem from a binary save file

This will read the problem saved by 'writeModel' from the file name set by 'modelFile'.

`-writeGSolution`

Puts glpk solution to file

Will write a glpk solution file to the given file name. It will use the default directory given by 'directory'. A name of '$' will use the previous value for the name. This is initialized to 'stdout' (this defaults to ordinary solution if stdout). If problem created from gmpl model - will do any reports.

`-writeModel`

save model to binary file

This will write the problem in binary foramt to the file name set by 'modelFile' for future use by readModel.

`-nextBestSolution`

Prints next best saved solution to file

To write best solution, just use writeSolution. This prints next best (if exists) and then deletes it. This will write a primitive solution file to the file name set by 'nextBestSolutionFile'. The amount of output can be varied using 'printingOptions' or 'printMask'.

-writeSolution

writes solution to file (or stdout)

This will write a primitive solution file to the file set by 'solFile'. The amount of output can be varied using 'printingOptions' or 'printMask'.

-solution

writes solution to file (or stdout) (synonym for writeSolution).

This will write a primitive solution file to the file set by 'solFile'. The amount of output can be varied using 'printingOptions' or 'printMask'.

-writeSolBinary

writes solution to file in binary format

This will write a binary solution file to the file set by 'solBinaryFile'. To read the file use fread(int) twice to pick up number of rows and columns, then fread(double) to pick up objective value, then pick up row activities, row duals, column activities and reduced costs - see bottom of ClpParamUtils.cpp for code that reads or writes file. If name contains '_fix_read_', then does not write but reads and will fix all variables

-writeStatistics

writes collected statistics to CSV file

This writes the statistics gathered so far to the file designated by csvStatistics (default 'stats.csv'). If no file name is supplied when the command is run, the previous CSV statistics file name is used.

-writeFeatures

writes instance features to CSV file

This extracts all OsiFeatures from the current MIP instance and appends them as a single row to the file designated by csvFeatures (default 'features.csv'). If no file name is supplied the previous value is used. The header row is written automatically when the file is new or empty. A total of 207 numeric features are extracted, covering: - Problem size: number of columns (variables) and rows (constraints), non-zeros, matrix density, columns-per-row ratio. - Variable types: counts and percentages of binary, general integer and continuous variables; unbounded variables. - Constraint classes: partitioning, packing, covering, cardinality, knapsack, integer knapsack, invariant knapsack, singleton, aggregation, precedence, variable-bound and bin-packing rows. - Objective and matrix statistics: min/max/mean/std-dev of non-zero coefficients, objective coefficients and right-hand-side values; column non-zero distribution (fraction of columns with >= k non-zeros for k = 1, 2, 4, ..., 4096). All features are computed in O(nz) time.

-checkSolution

Check LP/MIP solution feasibility and write validation report

Recomputes constraint and bound violations from scratch and writes a machine-readable report to the specified file (default 'sol_validation.txt'). Reports feasibility status, largest primal and dual errors, and identifies the constraint/variable with the largest violation.

-csvFeatures

sets file name for writing out instance features

Sets the file name used by writeFeatures. If name is not specified the previous value is used. Initialized to 'features.csv'. The header row listing all 207 feature names is written automatically when the file is new or empty; subsequent calls append a new row.

-csvStatistics

sets file name for writing out statistics

This appends statistics to given file name. If name is not specified, the previous value will be used. This is initialized to '', i.e. it must be set. Adds header if file empty or does not exist.

-exportFile

sets name for file to export model to

This will set the name of the model will be written to and read from. This is initialized to 'export.mps'.

-importFile

sets name for file to import model from

This will set the name of the model to be read in with the import command. This is initialized to 'import.mps'

-gmplSolutionFile

sets name for file to store GMPL solution in

This will set the name the GMPL solution will be written to and read from. This is initialized to 'gmpl.sol'.

-mipReadFile

sets name for file to read mip start from

This will set the name the model will be written to and read from. This is initialized to 'prob.mod'.

-modelFile

sets name for file to store model in

This will set the name the model will be written to and read from. This is initialized to 'prob.mod'.

-nextSolutionFile

sets name for file to store suboptimal solutions in

This will set the name solutions will be written to and read from. This is initialized to 'next.sol'.

-priorityFile

Name of file to import priorities from

Priorities will be read from the given file name. It will use the default directory given by 'directory'. The default name is priorities.txt and it cannot be a compressed file.File is in csv format with allowed headings - name, number, priority, direction, up, down, solution. Exactly one of name and number must be given.

-solFile

sets name for file to store solution in

This will set the name the solution will be saved to and read from. By default, solutions are written to 'opt.sol'. To print to stdout, use printSolution.

-solBinaryFile

sets name for file to store solution in binary format

This will set the name the solution will be saved to and read from. By default, binary solutions are written to 'solution.file'.use printSolution.

-directory

Set Default directory for import etc.

This sets the directory which import, export, saveModel, restoreModel etc. will use. It is initialized to the current directory.

-dirNetlib

Set directory where the netlib problems are.

This sets the directory where the netlib problems reside. One can get the netlib problems from COIN-OR or from the main netlib site. This parameter is used only when -netlib is passed to cbc. cbc will pick up the netlib problems from this directory. If cbc is built without zlib support then the problems must be uncompressed.

-errorsAllowed

Whether to allow import errors

The default is not to use any model which had errors when reading the mps file. Setting this to 'on' will allow all errors from which the code can recover simply by ignoring the error. There are some errors from which the code can not recover, e.g., no ENDATA. This has to be set before import, i.e., -errorsAllowed on -import xxxxxx.mps.

Values: `off`, `on` (default: `off`)

-basisIn

Import basis from bas file

This will read an MPS format basis file from the given file name. It will use the default directory given by 'directory'. A name of '$' will use the previous value for the name. This is initialized to '', i.e. it must be set. If you have libz then it can read compressed files 'xxxxxxxx.gz' or xxxxxxxx.bz2.

-basisOut

Export basis as bas file

This will write an MPS format basis file to the given file name. It will use the default directory given by 'directory'. A name of '$' will use the previous value for the name. This is initialized to 'default.bas'.

-basisFile

sets the name for file for reading/writing the basis

This will read an MPS format basis file from the given file name. It will use the default directory given by 'directory'. If no name is specified, the previous value will be used. This is initialized to '', i.e. it must be set. If you have libz then it can read compressed files 'xxxxxxxx.gz' or xxxxxxxx.bz2.

-paramFile

set name of file to import parametrics data from

This will read a file with parametric data from the given file name and then do parametrics. It will use the default directory given by 'directory'. A name of '$' will use the previous value for the name. This is initialized to '', i.e. it must be set. This can not read from compressed files. File is in modified csv format - a line ROWS will be followed by rows data while a line COLUMNS will be followed by column data. The last line should be ENDATA. The ROWS line must exist and is in the format ROWS, inital theta, final theta, interval theta, n where n is 0 to get CLPI0062 message at interval or at each change of theta and 1 to get CLPI0063 message at each iteration. If interval theta is 0.0 or >= final theta then no interval reporting. n may be missed out when it is taken as 0. If there is Row data then there is a headings line with allowed headings - name, number, lower(rhs change), upper(rhs change), rhs(change). Either the lower and upper fields should be given or the rhs field. The optional COLUMNS line is followed by a headings line with allowed headings - name, number, objective(change), lower(change), upper(change). Exactly one of name and number must be given for either section and missing ones have value 0.0.

-errorsAllowed

Whether to allow import errors

The default is not to use any model which had errors when reading the mps file. Setting this to 'on' will allow all errors from which the code can recover simply by ignoring the error. There are some errors from which the code can not recover e.g. no ENDATA. This has to be set before import i.e. -errorsAllowed on -import xxxxxx.mps.

Values: `off`, `on` (default: `off`)

-keepNames

Whether to keep names from import

It saves space to get rid of names so if you need to you can set this to off. This needs to be set before the import of model - so -keepnames off -import xxxxx.mps.

Values: `off`, `on` (default: `on`)

17 Parallelism

-racingLP

Number of threads for opportunistic parallel LP racing at root node

When set to a value > 0, the root LP relaxation is solved by racing multiple LP method configurations in parallel (dual simplex, primal with Idiot crash, primal with Sprint). The first to reach optimality wins and the others are aborted. This can significantly reduce root LP time for problems where the default method is not the fastest. A value of 3 races all three configurations. Set to 0 to disable.

Range: 0 to 8 (default: 0)

-threads

Number of threads to try and use

To use multiple threads, set threads to number wanted. It may be better to use one or two more than number of cpus available. If 100+n then n threads and search is repeatable (maybe be somewhat slower), if 200+n use threads for root cuts, 400+n threads used in sub-trees.

Range: -100 to 100000 (default: 0)

18 General

-help

Print out version, non-standard options and some help

This prints out some help to get a user started. If you're seeing this message, you should be past that stage.

-end

Stops execution

This stops execution; end, exit, quit and stop are synonyms.

-exit

Stops cbc execution

This stops the execution of Cbc, end, exit, quit and stop are synonyms

-quit

Stops cbc execution

This stops the execution of Cbc, end, exit, quit and stop are synonyms

-stop

Stops cbc execution

This stops the execution of Cbc, end, exit, quit and stop are synonyms

`-version`

Print version

`-cplexUse`

Whether to use Cplex!

If the user has Cplex, but wants to use some of Cbc's heuristics then you can! If this is on, then Cbc will get to the root node and then hand over to Cplex. If heuristics find a solution this can be significantly quicker. You will probably want to switch off Cbc's cuts as Cplex thinks they are genuine constraints. It is also probable that you want to switch off preprocessing, although for difficult problems it is worth trying both.

Values: `off`, `on` (default: `off`)

`-rankConflictType`

Formula for combining directional conflict degrees into a single score.

Controls how d0 (conflicts when x=0) and d1 (conflicts when x=1) are combined: sum: d0+d1 — total propagation power (default); min: min(d0,d1) — both directions must be strong; product: sqrt(d0*d1) — product score analog, rewards balance.

Values: `min`, `sum`, `product`

`-extra1`

Extra integer parameter 1

Range: −INT_MAX to INT_MAX (default: -1)

`-extra2`

Extra integer parameter 2

Range: −INT_MAX to INT_MAX (default: -1)

`-extra3`

Extra integer parameter 3

Range: −INT_MAX to INT_MAX (default: -1)

`-extra4`

Extra integer parameter 4

Range: −INT_MAX to INT_MAX (default: -1)

`-rootHeurSchedule`

Enable two-phase parallel root heuristic schedule

When set to 1, replaces the default root heuristic execution with a two-phase parallel schedule. Phase 1 runs optimized diving configurations in parallel (stops on first feasible solution). Phase 2 runs improvement heuristics (RINS, etc.) on the found solution. Use with -threads to set the number of parallel threads.

Range: 0 to 1 (default: 0)

`-rankConflict`

Weight for conflict-graph degree in strong branching sort-key (0 = disabled).

When positive, the conflict graph degree of binary variables is used to augment the pseudo-cost-based sort key that determines which candidates receive strong branching LP solves. Higher-degree variables (those whose branching triggers more propagations) are prioritized. The boost factor is (1 + weight * scaledScore), where scaledScore depends on rankConflictType and the per-trust scaling powers. Default 0.2 (enabled, sum formula). Set to 0.0 to disable. Typical useful range: 0.1 to 0.5.

Range: 0 to 100 (default: 0)

`-rankConflictPowerTrusted`

Scaling exponent for conflict score when pseudo-costs are trusted (sqrt = 0.5).

When pseudo-cost observations are sufficient (trusted), conflict information acts as a gentle tie-breaker. The raw conflict score is raised to this power before weighting: 0.5 = square root (default, mild nudge), 0.333 = cube root (very mild), 1.0 = linear (full influence even when trusted).

Range: 0 to 1 (default: 0)

`-rankConflictPowerUntrusted`

Scaling exponent for conflict score when pseudo-costs are untrusted (linear = 1.0).

When pseudo-cost observations are insufficient (untrusted), conflict information is given stronger influence. The raw conflict score is raised to this power: 1.0 = linear (default, full influence), 0.5 = square root (moderate).

Range: 0 to 1 (default: 0)

`-rankRange`

Weight for variable-range criterion 1/min(maxRange,ub-lb) in strong branching sort-key (0 = disabled).

When positive, the domain width of integer variables is used to augment the sort key that determines which candidates receive strong branching LP solves. Score = 1/min(rankRangeMax, ub-lb): binary [0,1] scores 1.0, domains >= rankRangeMax score 1/rankRangeMax (floor), preventing large/unbounded vars from collapsing to ~0. Applies to all integer variables (not just binary). The boost factor is (1 + weight * scaledScore). Default 0.0 (disabled). Typical useful range: 0.01 to 0.3.

Range: 0 to 100 (default: 0)

`-rankRangePowerTrusted`

Scaling exponent for range score when pseudo-costs are trusted (sqrt = 0.5).

When pseudo-cost observations are sufficient (trusted), range information acts as a gentle tie-breaker. The raw score 1/min(maxRange,ub-lb) is raised to this power: 0.5 = square root (default, mild nudge), 0.333 = cube root, 1.0 = linear.

Range: 0 to 1 (default: 0)

-rankRangePowerUntrusted

Scaling exponent for range score when pseudo-costs are untrusted (linear = 1.0).

When pseudo-cost observations are insufficient, range information is given stronger influence. 1.0 = linear (default), 0.5 = square root (moderate).

Range: 0 to 1 (default: 0)

-rankRangeMax

Cap on domain width for range criterion: score = 1/min(rankRangeMax, ub-lb). Default 10.

Variables with domain width >= rankRangeMax all receive the same floor score (1/rankRangeMax), preventing large or unbounded integer domains from collapsing to a near-zero range score. Binary [0,1] always scores 1.0 (unaffected). Default 10.0: domains of 10 or wider are treated equally (floor score = 0.1). Increase to give more differentiation among wider domains.

Range: 1 to 1e+30 (default: 0)

-rankObjCoeff

Weight for objective coefficient magnitude criterion |c_j|^power in strong branching sort-key (0 = disabled).

When positive, the absolute value of a variable's objective coefficient |c_j| is used to augment the sort key that determines strong branching candidate priority. Score = |c_j|^scalingPower. Variables not in the objective (c_j=0) receive no boost. Most useful for untrusted variables where pseudo-costs are unreliable; for trusted variables pseudo-costs already capture the objective coefficient implicitly. Default 0.0 (disabled). Typical useful range: 0.01 to 0.3.

Range: 0 to 100 (default: 0)

-rankObjCoeffPowerTrusted

Scaling exponent for obj-coeff score when pseudo-costs are trusted. Default 0.1 (very slow growth).

When pseudo-cost observations are sufficient, objective coefficient acts as a gentle tie-breaker. Score = |c_j|^power: 0.1 (default) gives c=100?1.58, c=10000?2.51. Use 0.05 for even milder effect, 0.2 for more influence.

Range: 0 to 1 (default: 0)

-rankObjCoeffPowerUntrusted

Scaling exponent for obj-coeff score when pseudo-costs are untrusted. Default 0.2.

When pseudo-cost observations are insufficient, objective coefficient is allowed more influence. 0.2 (default): c=100?2.51, c=10000?6.31. Use 0.1 to match trusted, or 0.5 for sqrt (moderate growth).

Range: 0 to 1 (default: 0)

-rankNonzeros

Weight for column non-zeros criterion nz^power in strong branching sort-key (0 = disabled).

When positive, the number of constraints a variable appears in is used to augment the sort key that determines strong branching candidate priority. Score = nz^scalingPower (default 4th-root, very slow growth). Variables appearing in many constraints propagate their fixing more broadly. Applies to all integer variables. Designed as a cheap tie-breaker. Default 0.0 (disabled). Typical useful range: 0.01 to 0.1.

Range: 0 to 100 (default: 0)

`-rankNonzerosPowerTrusted`

Scaling exponent for nz score when pseudo-costs are trusted (4th-root = 0.25).

When pseudo-cost observations are sufficient, nz information acts as a gentle tie-breaker. Score = nz^power: 0.25 = 4th root (default, very slow growth), 0.5 = sqrt, 1.0 = linear.

Range: 0 to 1 (default: 0)

`-rankNonzerosPowerUntrusted`

Scaling exponent for nz score when pseudo-costs are untrusted (sqrt = 0.5).

When pseudo-cost observations are insufficient, nz information is given slightly more influence. 0.5 = sqrt (default), 1.0 = linear.

Range: 0 to 1 (default: 0)

`-rankConflictMaxPercBin`

Maximum % of binary integer variables for the conflict ranker to activate (default 97).

The conflict-graph ranker is beneficial primarily on mixed-integer problems (where not all integer variables are binary). When the fraction of binary variables among all integer variables is >= this threshold, the ranker is automatically disabled to avoid performance regressions on near-pure-binary instances. Set to 100 to always activate regardless of binary fraction.

Range: 0 to 100 (default: 97)

`-netlibBarrier`

Solve entire netlib test set with barrier

This exercises the unit test for clp and then solves the netlib test set using barrier. The user can set options before e.g. clp -kkt on -netlib

`-netlibDual`

Solve entire netlib test set (dual)

This exercises the unit test for clp and then solves the netlib test set using dual. The user can set options before e.g. clp -presolve off -netlib

`-netlib`

Solve entire netlib test set

This exercises the unit test for clp and then solves the netlib test set using dual or primal. The user can set options before e.g. clp -presolve off -netlib

-netlibPrimal

Solve entire netlib test set (primal)

This exercises the unit test for clp and then solves the netlib test set using primal. The user can set options before e.g. clp -presolve off -netlibp

-netlibTune

Solve entire netlib test set with 'best' algorithm

This exercises the unit test for clp and then solves the netlib test set using whatever works best. I know this is cheating but it also stresses the code better by doing a mixture of stuff. The best algorithm was chosen on a Linux ThinkPad using native cholesky with University of Florida ordering.